Task: Credit Transparency & Blast Credit Gating

Assigned by: Brij
Type: Frontend coding — React + TypeScript (lovable-frontend)
Estimated time: 6–10 hours
Branch name: `feat/credit-transparency`

What this task is

Right now a creator can walk into the blast tool, target 2,000 fans, hit send — and have no idea it'll cost more credits than they have. The blast starts, burns through their remaining credits, then stops partway through. Nobody told them this was going to happen. They find out when half their fans got the message and half didn't.

Your job is to make the credit system visible and protective at every step:

1. Tell users how much a blast will cost **before** they send it
2. **Block the send** if they can't afford it — don't let them start something they can't finish
3. **Suggest a fix** — "you can only afford 400 recipients, here's how to narrow your audience"
4. Explain what credits are and what happens when they're gone, in plain English, wherever it's relevant
5. Fix a few small annoyances (billing hidden from the menu, credits invisible on mobile)

This is entirely frontend work. You're not touching the backend — all the data you need is already coming from the API.

Background: how credits work

Read this first so the UI copy you write is accurate.

- Every SMS sent = **1 credit per SMS segment**
- A short message (≤ 160 characters) = **1 segment = 1 credit per recipient**
- A message between 161–306 characters = **2 segments = 2 credits per recipient**
- A message over 306 characters = **3 segments = 3 credits per recipient**
- Credits are allocated monthly by plan and **reset on the billing renewal date**
- Unused monthly credits **do not roll over**
- Booster credits (one-time purchases) **never expire**
- When credits run out: currently the AI bot keeps responding (we're fixing this separately), but **blasts stop mid-send**. For now, just be honest in the UI that a blast will stop if credits are exhausted mid-send.

The billing API already tells the frontend:

- `credits_used` — how many used this period
- `credits_total` — total allowed this period
- `credits_warning` — `"low"` (>75% used) or `"critical"` (>90% used) or null
- `unlimited` — true/false

The blast composer already knows:

- `body` — the message text (so you can calculate segment count)
- `previewCount` — the number of recipients (fetched when user clicks "Preview Audience")

So: `estimated_credits = previewCount × segments(body.length)` — you have everything you need.

The files you'll be working in

All files are inside `lovable-frontend/src/`.

File	What it is
`components/blast/BlastConfirmDialog.tsx`	The modal that appears before sending — **main place to add credit check**
`pages/BlastTool.tsx`	The blast compose page — add cost estimate inline near the send button
`pages/Usage.tsx`	Credit usage page — add explanatory copy
`pages/Billing.tsx`	Billing page — also add explanatory copy
`components/shell/CreditsWidget.tsx`	The header chip (`CreditsChip`) and floating widget (`CreditsWidget`)
`components/dashboard/UserMenu.tsx`	The user dropdown — add Billing link here
`types/billing.ts`	TypeScript types — you may need to read this for reference

---

The 6 things to build

Work through these in order — each one is self-contained.

---

1. Credit cost estimate in the blast confirm dialog

File: `components/blast/BlastConfirmDialog.tsx`

The confirm dialog already shows audience, recipient count, channel, and message. Add a **"Credit cost"** row showing:

- Estimated credits this blast will use
- How many you currently have remaining
- Whether you can afford it

How to calculate:

```
function segmentCount(bodyLength: number): number {
  if (bodyLength <= 160) return 1;
  if (bodyLength <= 306) return 2;
  return 3;
}

const estimatedCost = previewCount !== null
  ? previewCount * segmentCount(body.length)
  : null;
```

You'll need to pass `creditsRemaining` into the dialog. The dialog is opened from `BlastTool.tsx` — find where `<BlastConfirmDialog` is rendered, fetch/pass the billing status in there.

What to show in the dialog:

- If `estimatedCost` is null (no preview count yet): don't show the row
- If `estimatedCost <= creditsRemaining`: show in green/neutral — "~{estimatedCost.toLocaleString()} credits · {creditsRemaining.toLocaleString()} remaining ✓"
- If `estimatedCost > creditsRemaining` but not by much (< 20% over): show in yellow — "~{estimatedCost.toLocaleString()} credits needed · only {creditsRemaining.toLocaleString()} remaining"
- If `estimatedCost > creditsRemaining` significantly: show in red — same message, red text

---

2. Block the send when credits are insufficient

File: `components/blast/BlastConfirmDialog.tsx`

Currently the Send button is disabled only when `previewCount === null || previewCount <= 0`.

Add a second disable condition: also disable when `estimatedCost > creditsRemaining`.

When disabled due to credits (not due to no audience preview), change the button text from "Run Preview Audience first" to something like "Not enough credits — top up to send".

The button should link to `/usage` or show a CTA inline — don't just dead-end the user.

---

3. "Suggest a smaller audience" when over the credit limit

File: `components/blast/BlastConfirmDialog.tsx` (and optionally `pages/BlastTool.tsx`)

When `estimatedCost > creditsRemaining`, below the credit row in the confirm dialog, show a suggestion block:

> "Your credits cover about **{affordableCount.toLocaleString()} recipients**.
> Consider sending to your **most engaged fans** instead."
> [Send to Top Fans →] [Buy Credits →]

Where:

```
const affordableCount = Math.floor(creditsRemaining /
segmentCount(body.length));
```

"Send to Top Fans →" should close the dialog and update the blast audience to Smart Send / Top N Engaged (check how `SmartSendPreview` works in `components/blast/SmartSendPreview.tsx`). If that's complex, just close the dialog and show a toast: "Try the Smart Send option to target your most engaged fans."

"Buy Credits →" should navigate to `/billing` in a new tab.

---

4. Inline cost estimate in the compose panel (before the confirm dialog)

File: `pages/BlastTool.tsx` — inside `ComposePanel`, near the Send/Schedule buttons

Once the user has run "Preview Audience" and the `previewCount` is known, show a small line near the send button:

> ⚡ Estimated cost: ~{estimatedCost.toLocaleString()} credits

Keep it small and subtle — just `text-xs text-muted-foreground`. This lets users see the cost while still composing, not just at the confirmation step.

If credits are critically low and the blast would exceed them, show it in yellow/red here too so they notice before they even click send.

---

5. Fix the two small discoverability issues

5a — Add Billing to the user menu

File: `components/dashboard/UserMenu.tsx`

Find where the menu items are rendered. Add a "Billing" link pointing to `sp("/billing")`. Put it between Usage and the logout/separator. Simple nav item, same style as the others.

5b — Make the credits chip visible on mobile

File: `components/shell/CreditsWidget.tsx`

Find `CreditsChip`. It has `hidden md:inline-flex` which hides it on mobile. Either:

- Remove the `hidden md:` so it shows on all screen sizes (check if it fits), OR
- Add it to the mobile nav/menu so mobile users can still see their credit balance

Pick whichever looks better on a phone.

---

6. Explanatory copy — what credits are and what happens when they run out

Files: `pages/Usage.tsx`, `pages/Billing.tsx`

The current Usage page has a small tooltip that says:

> "Credits are counted by SMS segments. Short messages (≤160 chars) = 1 credit. Longer messages = 2–3 credits. MMS blasts with a photo = 3 credits per fan. Credits reset monthly."

That's good for a tooltip. But we also need more visible explanatory sections.

On `Usage.tsx` — below the credit meter card, add a small "How credits work" section. Keep it simple:

> **How credits work**
> Every SMS you send uses 1 credit per recipient for short messages (under 160 characters). Longer messages use 2–3 credits per person. Your credits reset on your renewal date each month.
>
> **What happens when credits run out?**
> Active blasts will stop mid-send. AI replies to fans will continue (we alert you before this happens). You can top up anytime with a credit booster — they never expire.

On `Billing.tsx` — already has a credit bar. Below the period reset date, add one sentence: "Credits reset each billing cycle. Unused monthly credits do not roll over, but booster credits never expire."

Keep the copy tight. Don't over-explain.

Definition of done

- [ ] Blast confirm dialog shows estimated credit cost for the blast
- [ ] Blast confirm dialog color-codes the cost (green = fine, yellow = close, red = over limit)
- [ ] Send button disabled and relabeled when credits are insufficient
- [ ] "Suggest smaller audience" block appears when user can't afford the blast, with affordableCount shown
- [ ] Inline cost estimate visible in compose panel after audience preview runs
- [ ] Billing link added to user menu
- [ ] Credits chip visible on mobile (or added to mobile menu)
- [ ] "How credits work" + "What happens when they run out" copy added to Usage page
- [ ] One-liner clarification added to Billing page
- [ ] No TypeScript linter errors introduced
- [ ] Tested in mock mode (look for `mockMode` — the app has a preview mode you can trigger)

How to test it locally

```
cd lovable-frontend
npm install
npm run dev
```

The app runs against the real operator API. If you don't have API access, set `VITE_MOCK_MODE=true` (or look at how `mockMode` is set — the app has a built-in preview mode, check `useAuthGuard` in `components/dashboard/DashboardShell.tsx`).

To test the credit-over-limit state: in `pages/BlastTool.tsx` temporarily hardcode a small `creditsRemaining` value (like `50`) to simulate being almost out. Then compose a blast with a 200-character message targeting 100 recipients — that's 200 credits estimated, over your fake limit of 50.

What good looks like

The user experience should be:

1. User selects audience → clicks "Preview Audience" → sees "450 recipients"
2. User sees inline: "⚡ ~450 credits estimated · 820 remaining" → everything's fine, proceed
3. Different scenario: user sees "⚡ ~900 credits estimated · 820 remaining" → yellow warning inline
4. User clicks Send → confirm dialog opens → big clear red block: "You need 900 credits but only have 820 remaining. Your blast will stop after ~820 sends."
5. Suggestion: "Send to your top 410 engaged fans instead" + "Buy Credits" button
6. Send button says "Not enough credits — top up to send" and is disabled

This should feel like a guard rail, not a punishment. The tone should be helpful: "here's what you can do" not "you can't do that."

Submitting

Open a PR against `main` with branch `feat/credit-transparency`. Fill out the PR description with what you built and any design decisions you made. Tag Brij for review.